Reverse Engineering Windows Kernel Mode Drivers

Supporting Material - SteelCon 2025 Workshop / Michael Rowley / https://michaelrowley.dev

Introduction

Learn Your Tool

- G1: Ghidra
- B1: Binary Ninja (Cloud)

Exploring DriverEntry (Entrypoint)

- G2: Ghidra
- B2: Binary Ninja

IoCreateDevice

- G3: Ghidra
- B3: Binary Ninja

Interrupt Request Packets (IRPs)

- G4: Ghidra
- B4: Binary Ninja

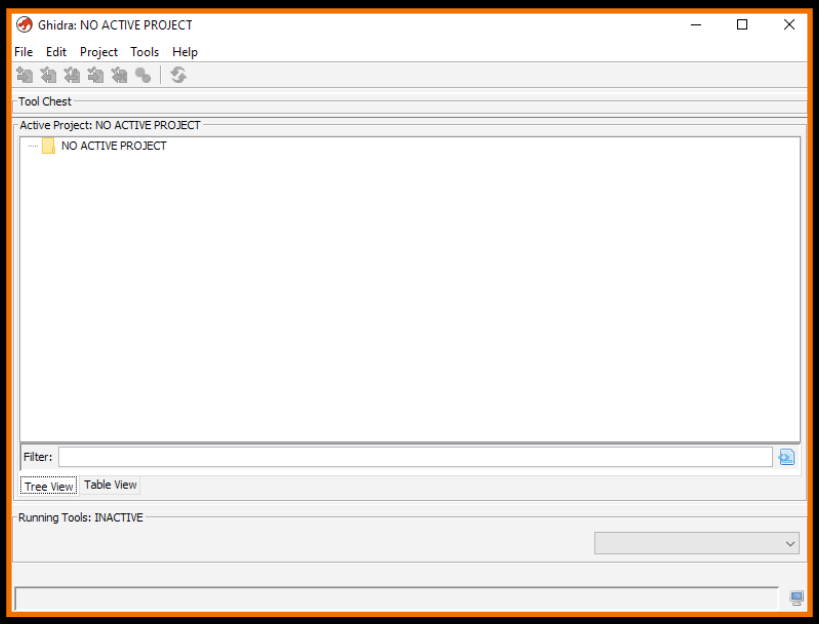
Interrupt Request Packet - Major Function Handlers

- G5: Ghidra
- B5: Binary Ninja

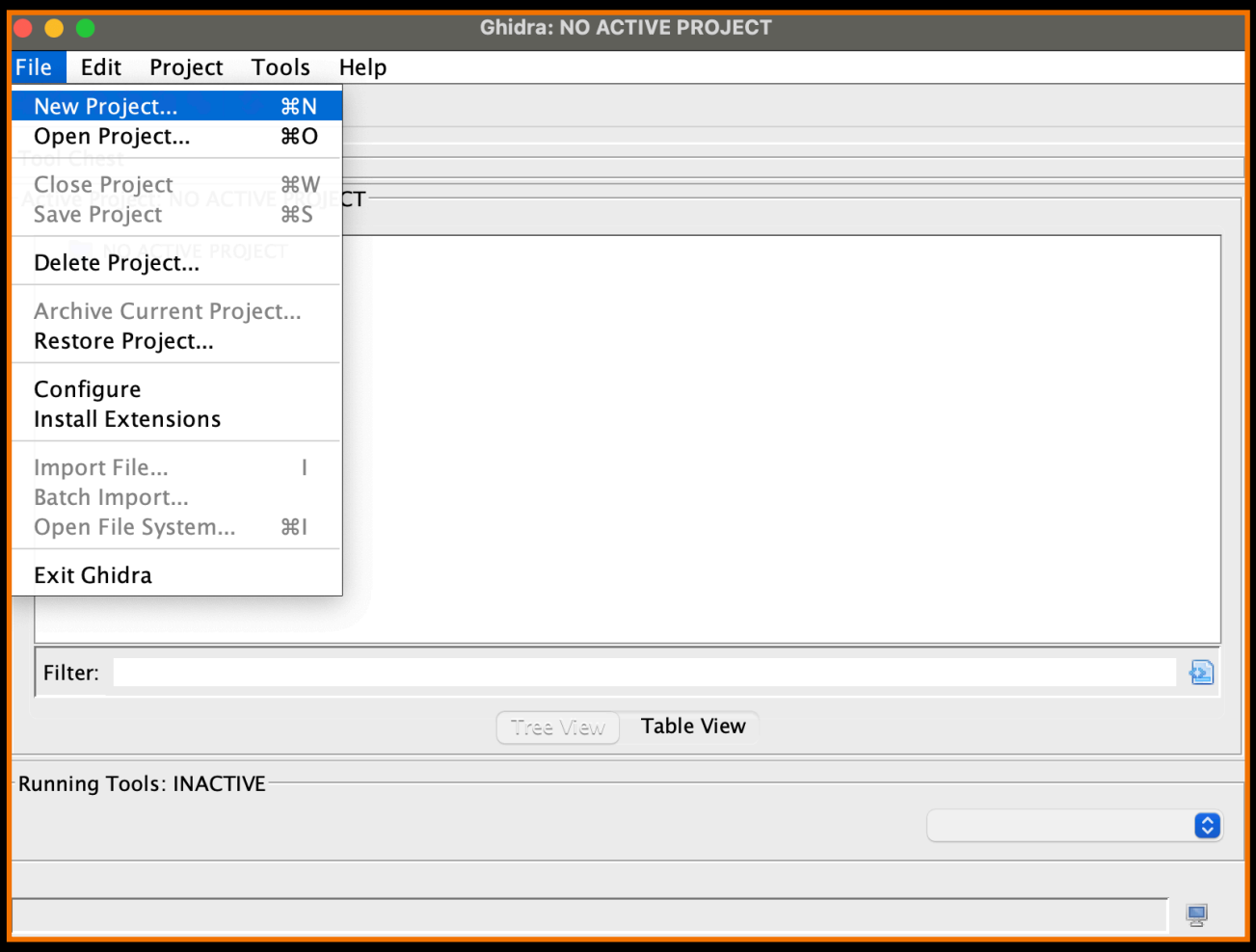
Final Challenge: Triaging Real Vulnerabilities

- Easy: Beep
- Medium: MSI Afterburner
- Hard: AMD Crash Defender
- Hard: HTTP Debugger

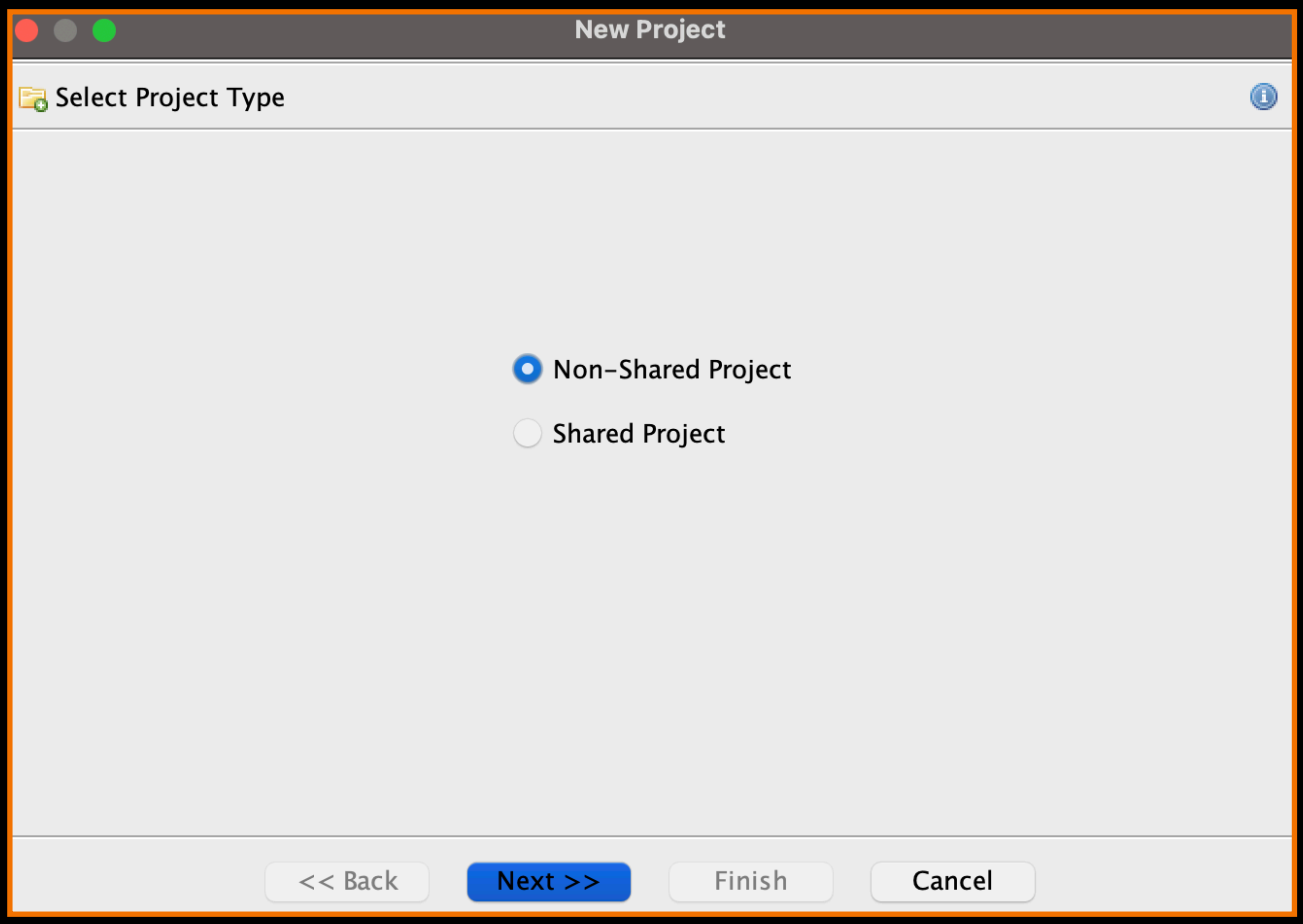
Introduction
Testing
Learn Your Tool
G1: Ghidra



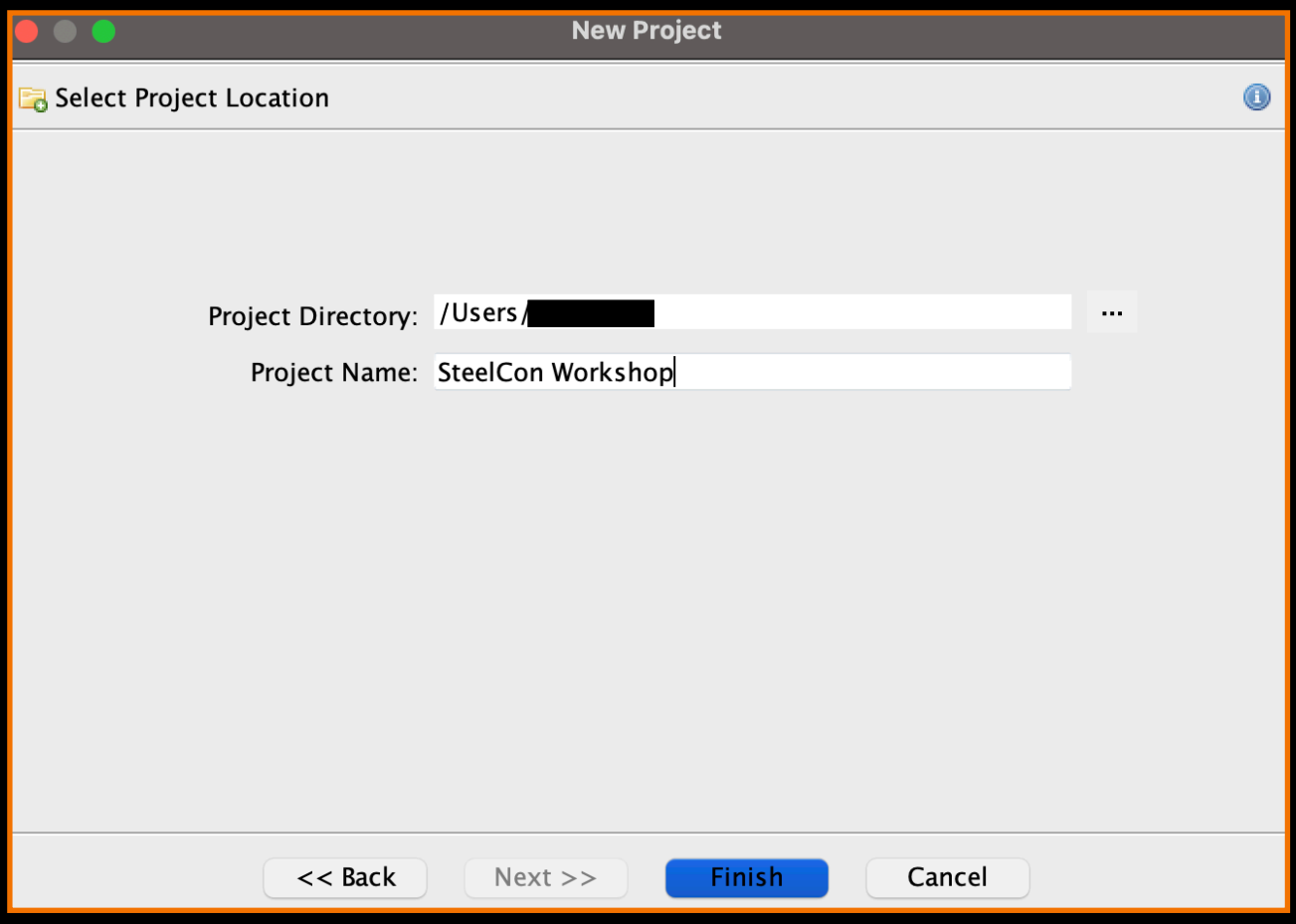
1. Launch Ghidra
2. Create a new project (File > X)



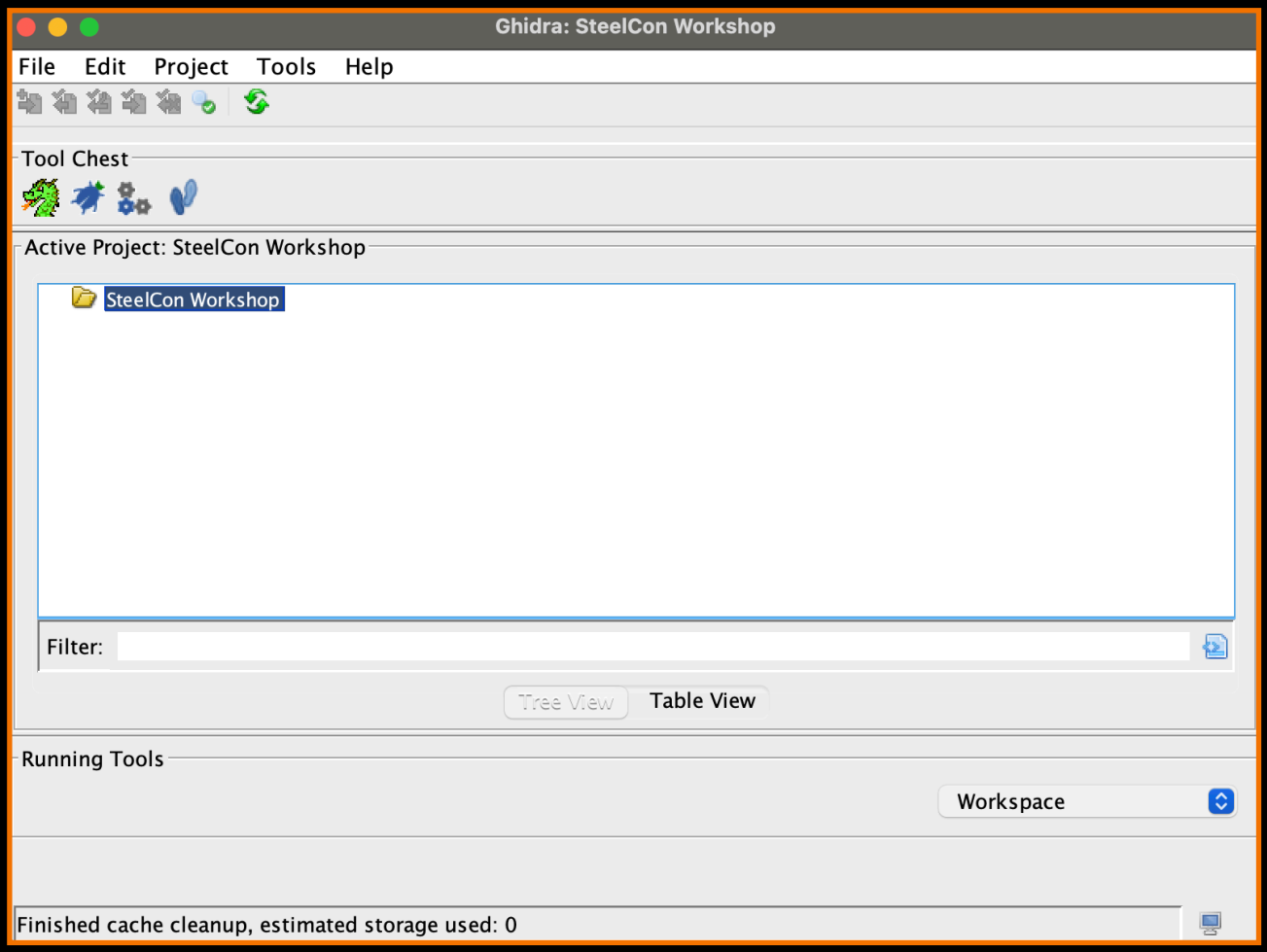
i. You'll want to keep the "Non-Shared Project" option selected as we aren't collaborating over a Ghidra Server instance



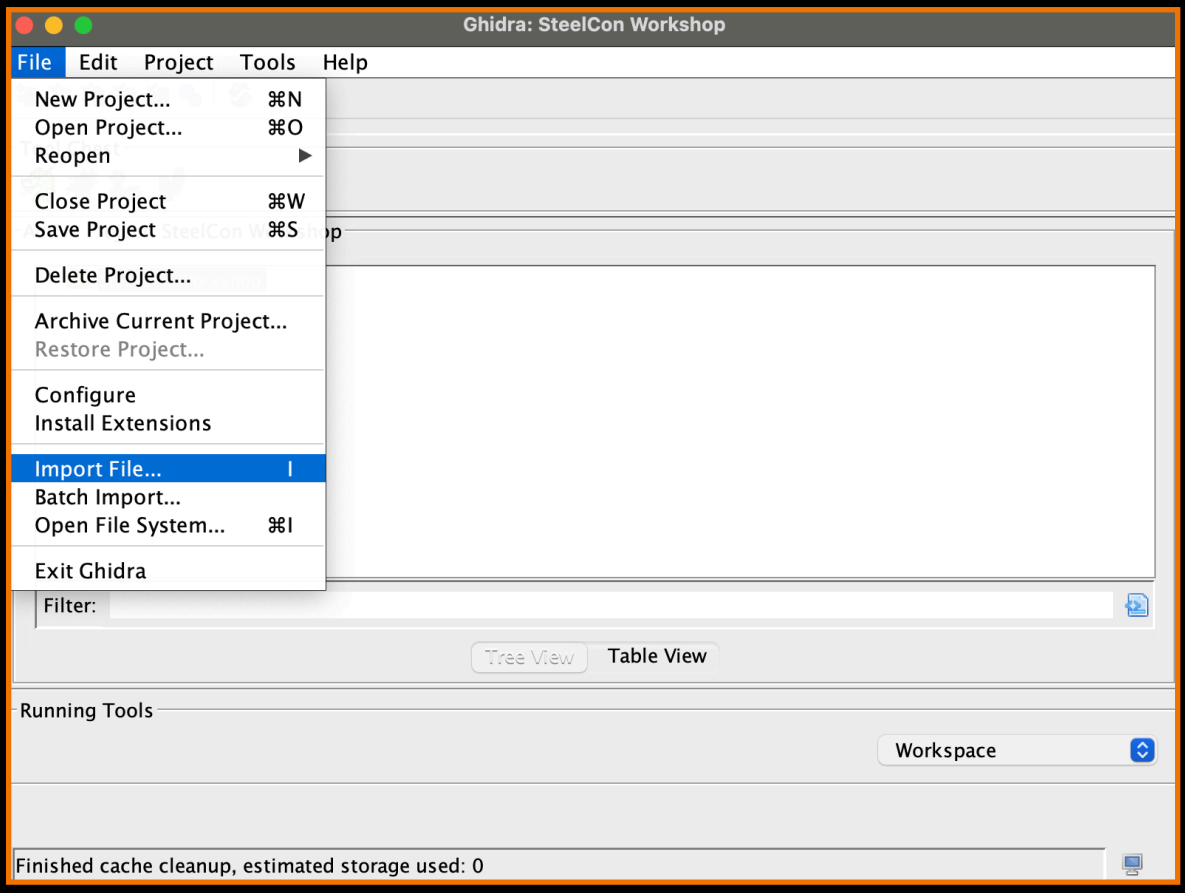
ii. The name for your project doesn't matter, just make sure that you keep it in a stable location



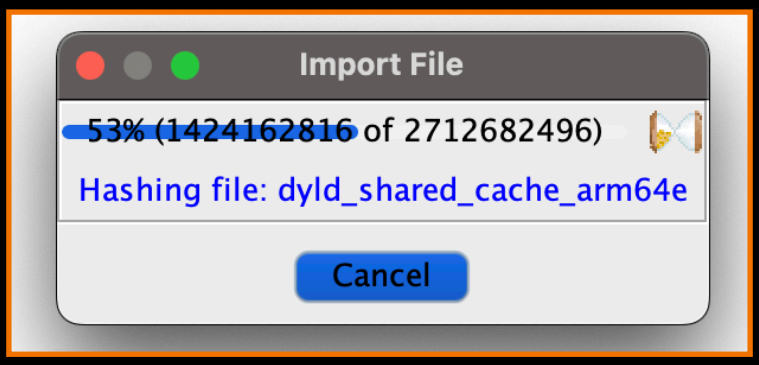
You should find yourself with a window like this:



3. Now that you have a project, import the beep.sys file to your workspace

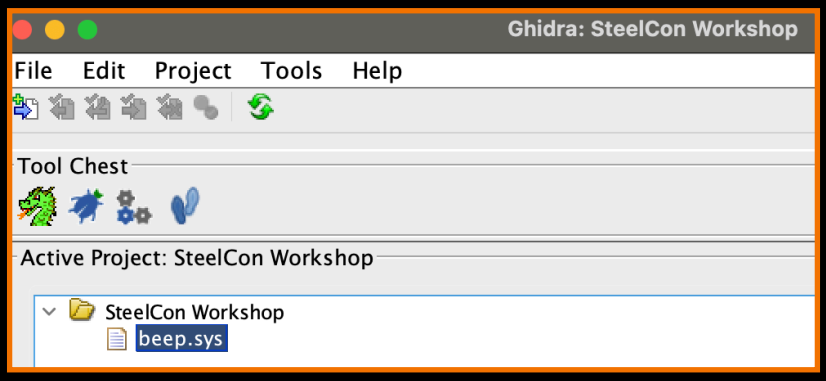


i. Leave the default import settings

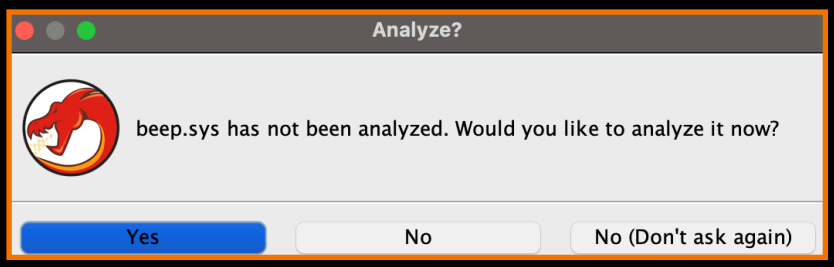


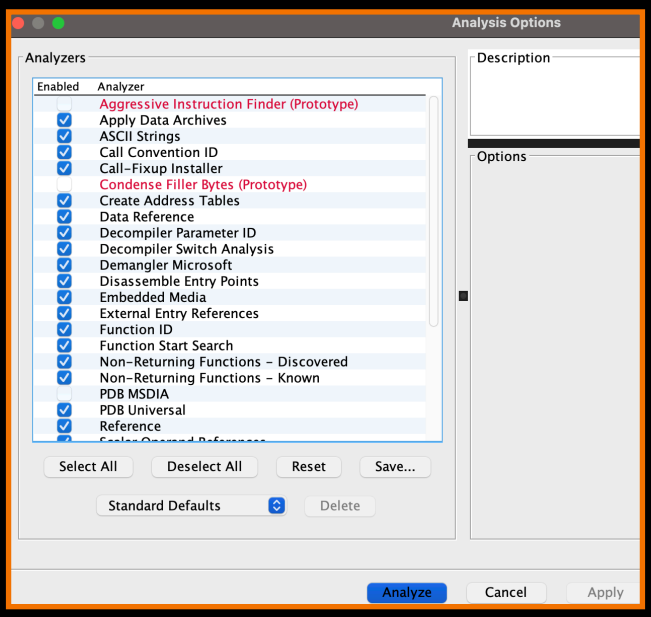
ii. The actual import process should take 30s at most

4. Open (double click) your sample in the workspace

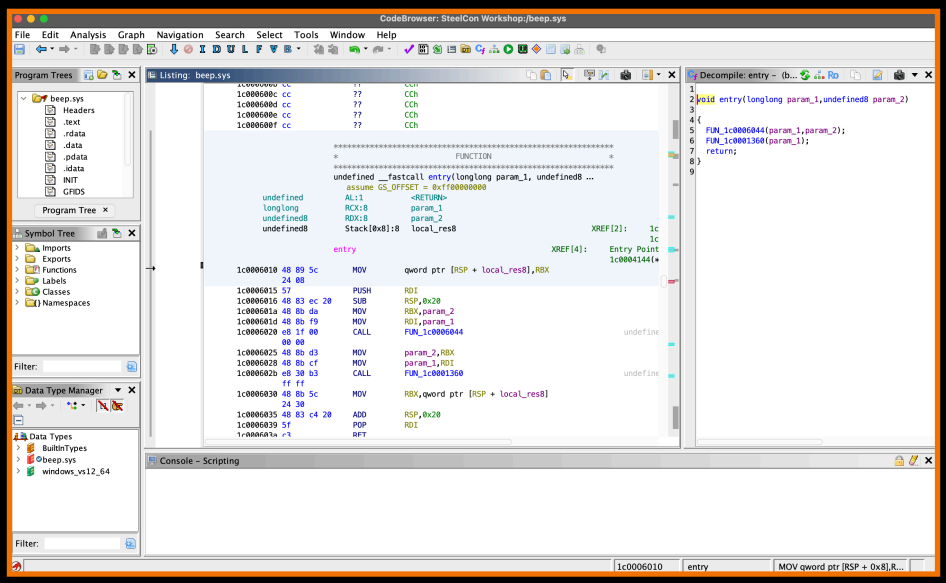


5. Opt to analyse the file now using Ghidra's default settings

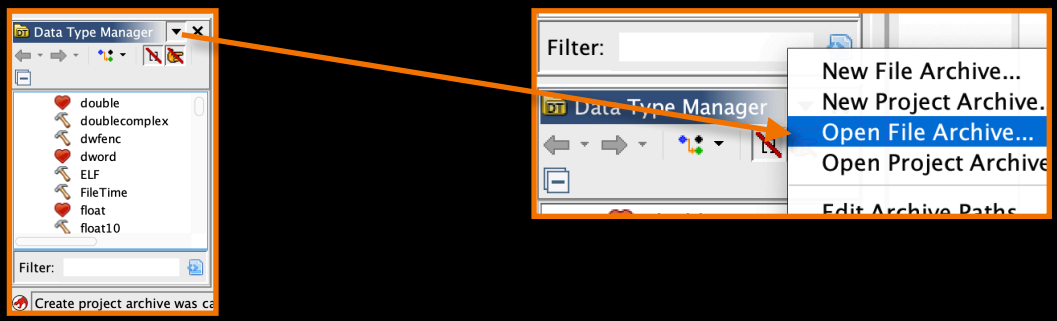




6. Ghidra will position the disassembly listing at the entrypoint of the binary, and spawn a number of ‘windows’ showing different types of information:

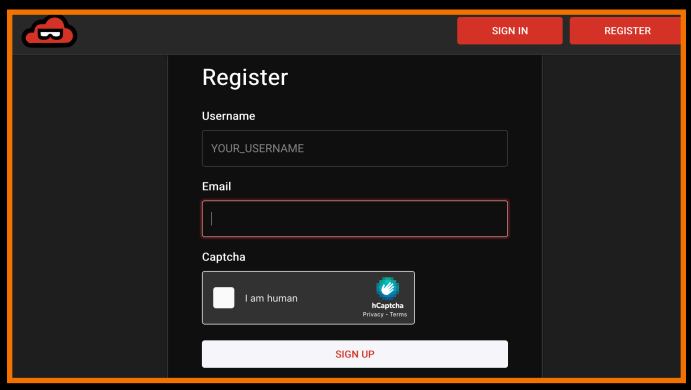


- Listing
  - Shows the virtual memory offsets/addresses of instructions and image data
  - Executable code is represented as assembly (in this case, x86)
- Decompile
  - Synchronised to the current ‘listing’ context (clicking a different function in the ‘listing’ window will change the content shown in ‘decompile’)
  - Provides a pseudo-C view, which is easier to digest than assembly in most cases (falling back to assembly is a good skill, as Ghidra’s decompiler can be inaccurate at times)
- Program Trees
  - Shows the different sections of the binary image
- Symbol Tree
  - Lists all derived symbols (imports, exports, internal functions, etc.) that Ghidra could infer/extract from the binary
- Data Type Manager
  - Lists all of the data types that are relevant to this sample; some that Ghidra included from a common Windows library set, some unnamed structures from the binary, and some ‘built in’ ones (booleans, integers, etc.)
    - ***<u>You MUST add the Windows kernel data types</u>*** by downloading this <u>file</u> and opening it in this window:

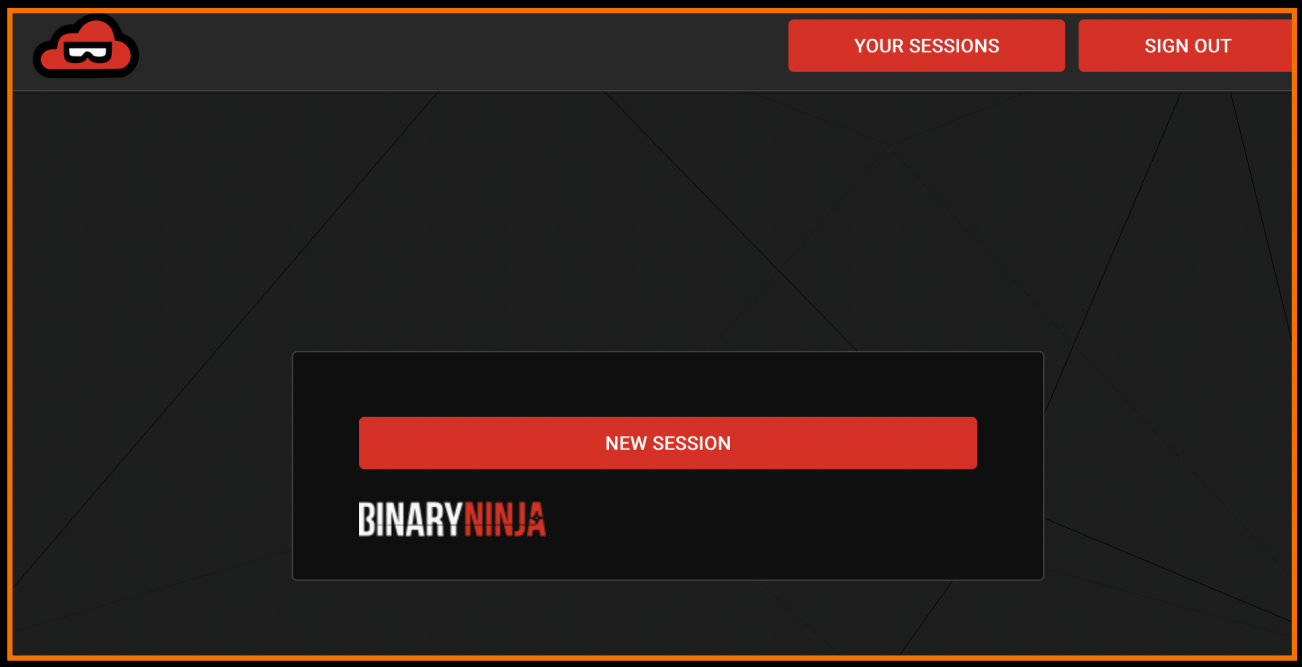


B1: Binary Ninja (Cloud)

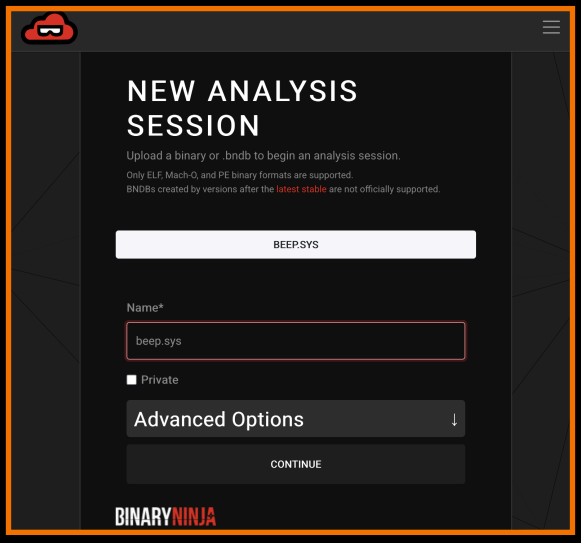
1. Visit https://cloud.binary.ninja/
2. Sign up to Binary Ninja Cloud (developed by Vector35)



3. Check the email and verify your account
4. Log in to your account
5. Create a new session



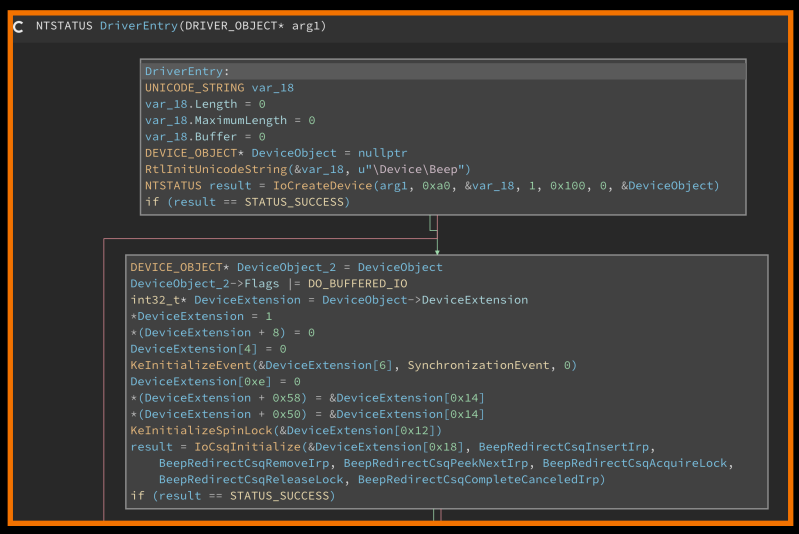
6. Upload the ‘beep.sys’ sample



At this point, you should arrive to the entrypoint identified by Binary Ninja:
Explore each of the different modes! They show different information that can be used to identify flaws/context about the driver.



- Disassembly
  - Shows a directed graph view of the application's possible execution flow(s)
  - Switch between Assembly, a number of intermediate representations (IRs), and pseudo C/ Rust



- Strings
  - Shows all of the ASCII/Unicode strings that could be extracted from the binary, this is useful for identifying interesting functionality and locating debug functions

```
00000001c000004d !This program cannot be run in DOS mode.\r\r\n$
00000001c00000c8 Rich
00000001c00001e8 .text
00000001c000020f h.rdata
00000001c0000237 H.data
00000001c0000260 .pdata
00000001c0000287 H.idata
00000001c00002af HINIT
00000001c00002d7 bGFIDS
00000001c00002ff B.rsrc
00000001c0000327 B.reloc
00000001c0001019 \nH9Q
00000001c0001072 \\$ VWAWH
00000001c00010ac W E3
00000001c00010c0 L$@H
00000001c000110b \\$X3
00000001c0001113 A__^
00000001c0001144 T$ H
00000001c0001167 T$ H
00000001c000118c \\$@H
00000001c0001191 l$HH
00000001c0001196 t$PH
00000001c0001211 \\$03
00000001c0001237 t$ WH
```

- Types
  - Structures that Binary Ninja has identified as being used by the sample file

```
                    struct CM_Power_Data_s
                    {
0000000000000000        ULONG PD_Size;
0000000000000004        DEVICE_POWER_STATE PD_MostRecentPowerState;
0000000000000008        ULONG PD_Capabilities;
000000000000000c        ULONG PD_D1Latency;
0000000000000010        ULONG PD_D2Latency;
0000000000000014        ULONG PD_D3Latency;
0000000000000018        DEVICE_POWER_STATE PD_PowerStateMapping [0x7];
0000000000000034        SYSTEM_POWER_STATE PD_DeepestSystemWake;
                    };

                    struct COFF_Header
                    {
0000000000000000        char magic [0x4];
0000000000000004        enum coff_machine machine;
0000000000000006        uint16_t numberOfSections;
0000000000000008        uint32_t timeDateStamp;
000000000000000c        uint32_t pointerToSymbolTable;
0000000000000010        uint32_t numberOfSymbols;
0000000000000014        uint16_t sizeOfOptionalHeader;
0000000000000016        enum coff_characteristics characteristics;
                    };
```

- Linear Disassembly
  - Textual (non-graph) version of the Disassembly view

```
NTSTATUS DriverEntry(DRIVER_OBJECT* arg1)

UNICODE_STRING var_18
var_18.Length = 0
var_18.MaximumLength = 0
var_18.Buffer = 0
DEVICE_OBJECT* DeviceObject = nullptr
RtlInitUnicodeString(&var_18, u"\Device\Beep")
NTSTATUS result = IoCreateDevice(arg1, 0xa0, &var_18, 1, 0x100, 0, &DeviceObject)

if (result == STATUS_SUCCESS)
    DEVICE_OBJECT* DeviceObject_2 = DeviceObject
    DeviceObject_2->Flags |= DO_BUFFERED_IO
    int32_t* DeviceExtension = DeviceObject->DeviceExtension
    *DeviceExtension = 1
    *(DeviceExtension + 8) = 0
    DeviceExtension[4] = 0
    KeInitializeEvent(&DeviceExtension[6], SynchronizationEvent, 0)
    DeviceExtension[0xe] = 0
    *(DeviceExtension + 0x58) = &DeviceExtension[0x14]
    *(DeviceExtension + 0x50) = &DeviceExtension[0x14]
    KeInitializeSpinLock(&DeviceExtension[0x12])
    result = IoCsqInitialize(&DeviceExtension[0x18], BeepRedirectCsqInsertIrp,
        BeepRedirectCsqRemoveIrp, BeepRedirectCsqPeekNextIrp, BeepRedirectCsqAcquireLock,
        BeepRedirectCsqReleaseLock, BeepRedirectCsqCompleteCanceledIrp)

    if (result == STATUS_SUCCESS)
        DEVICE_OBJECT* DeviceObject_1 = DeviceObject
        arg1->DriverStartIo = BeepStartIo
        arg1->DriverUnload = BeepUnload
        arg1->MajorFunction[0] = BeepOpen
```

- Triage
  - Abstract information about the file

Entropy



Headers

```
Type:                PE 64-bit
Machine:             AMD64
Subsystem:           NATIVE
Timestamp:           Sat Aug 12 19:19:51 2051
Image Base:          0x1c0000000
Entry Point:         0x1c0006010
Section Alignment:   0x1000
File Alignment:      0x1000
Checksum:            0x00014859
Base of Code:        0x1c0001000
Size of Code:        0x2000
Size of Init Data:   0x7000
Size of Uninit Data: 0x0
Size of Headers:     0x1000
Size of Image:       0xa000
Stack Size:          0x1000 / 0x40000
Heap Size:           0x1000 / 0x100000
Linker Version:      14.30
Image Version:       10.00
OS Version:          10.00
Subsystem Version:   10.00
COFF Characteristics: EXECUTABLE_IMAGELARGE_ADDRESS_AWARE
DLL Characteristics:  HIGH_ENTROPY_VADYNAMIC_BASENX_COMPATGUARD_CF
```

Exploring DriverEntry (Entrypoint)

For this task, you'll be exploring the contents of the DriverEntry function which serves as the 'entrypoint' for execution of Windows drivers.

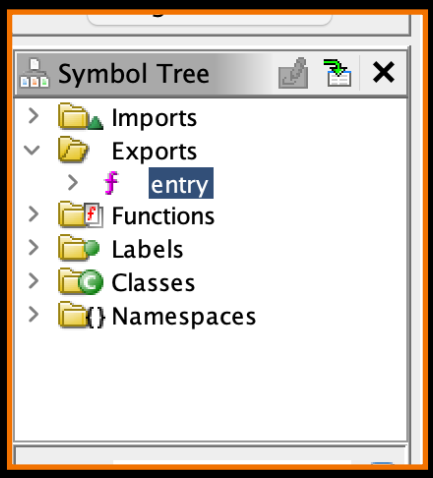
This can be as simple or complex as you make it, as the goal for you is to first locate and navigate to the entrypoint (simple) before 'annotating' it (simple/complex, depending on how you feel!)

When we refer to 'annotations' the goal is for you to add comments to the reverse engineered view, rename variables so that they make logical sense, and (as a bonus if you have time) discover how to create/populate custom structures and assign variable types to them.

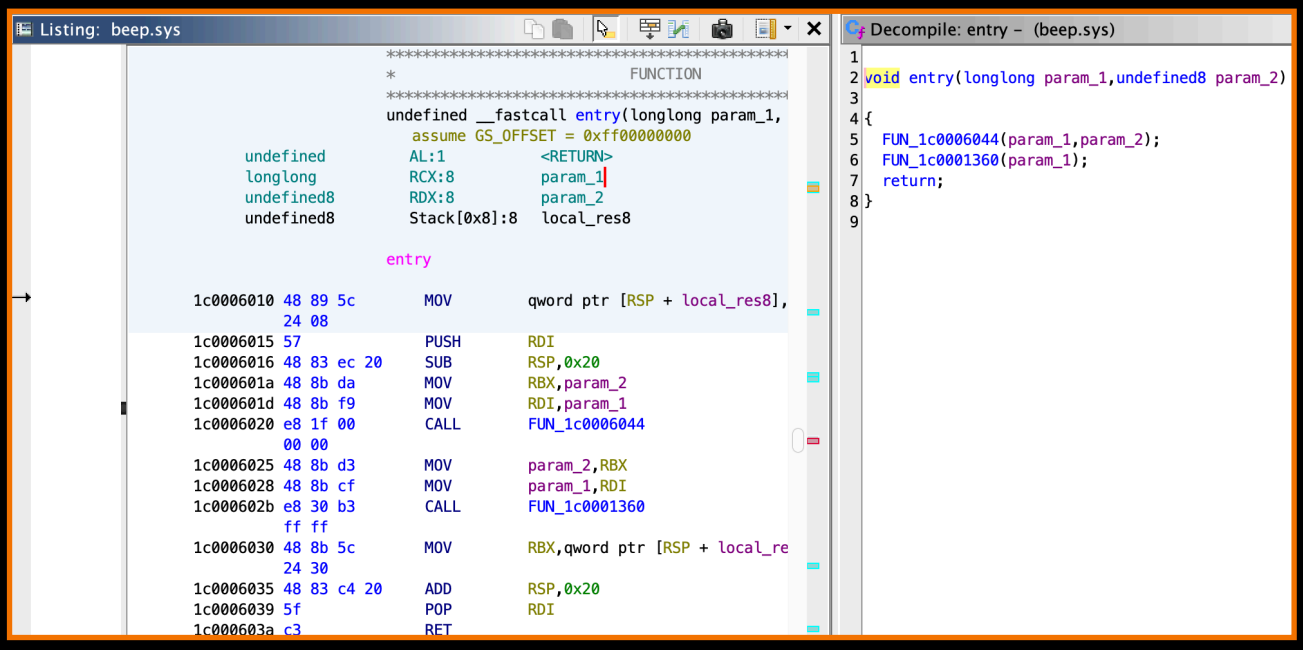
Continue using beep.sys (previously imported/loaded in the last section) for this.

G2: Ghidra

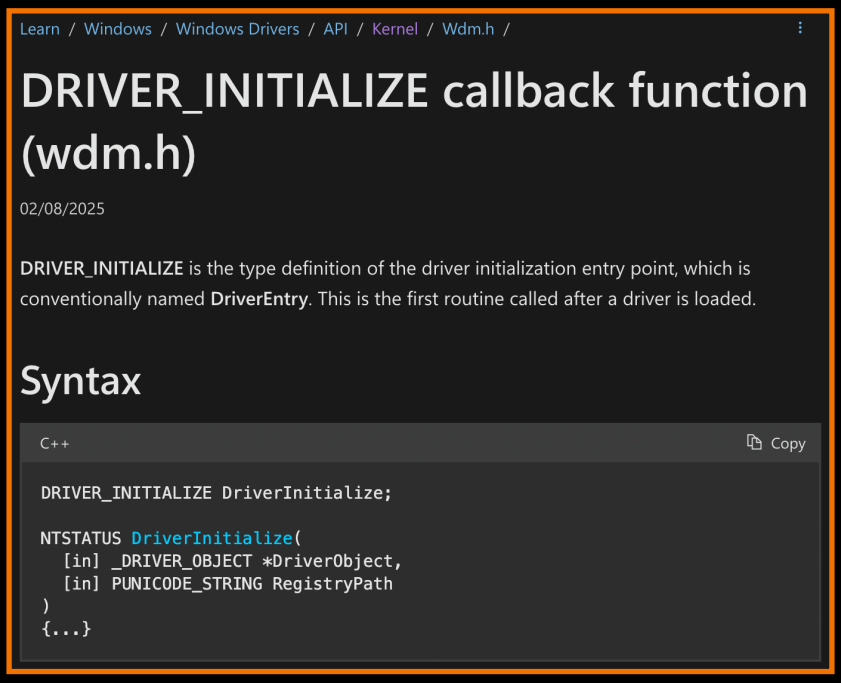
1. Find DriverEntry (in Ghidra, this is marked as 'entry' by default):
2. Consider what the entrypoint *does* (hint: it calls two functions)



3. With this information, we'll want to 'annotate' the decompiled function to add much-needed context.



i. Firstly, Ghidra isn't 'KMDF-aware' (KMDF=Kernel Mode Driver Framework), it doesn't understand the structure that DriverEntry takes. We can use Microsoft's documentation to add context here.



Learn / Windows / Windows Drivers / API / Kernel / Wdm.h /

DRIVER_INITIALIZE callback function (wdm.h)

02/08/2025

DRIVER_INITIALIZE is the type definition of the driver initialization entry point, which is conventionally named **DriverEntry**. This is the first routine called after a driver is loaded.

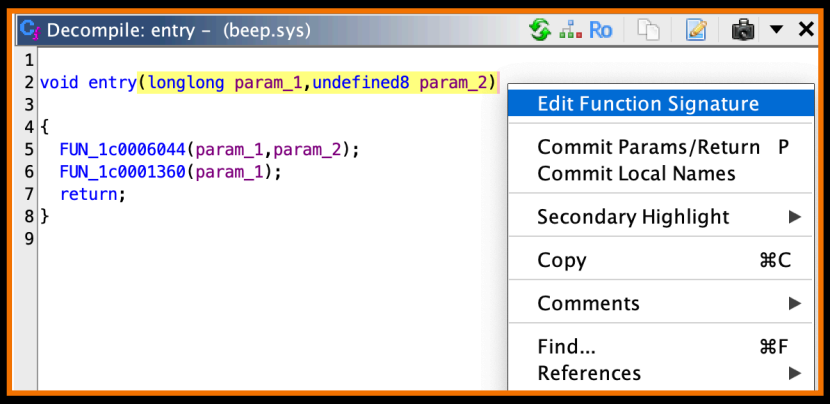
Syntax

C++ Copy

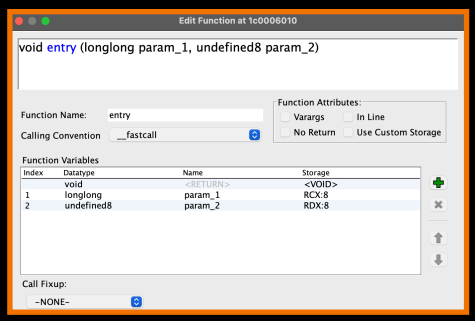
```
DRIVER_INITIALIZE DriverInitialize;

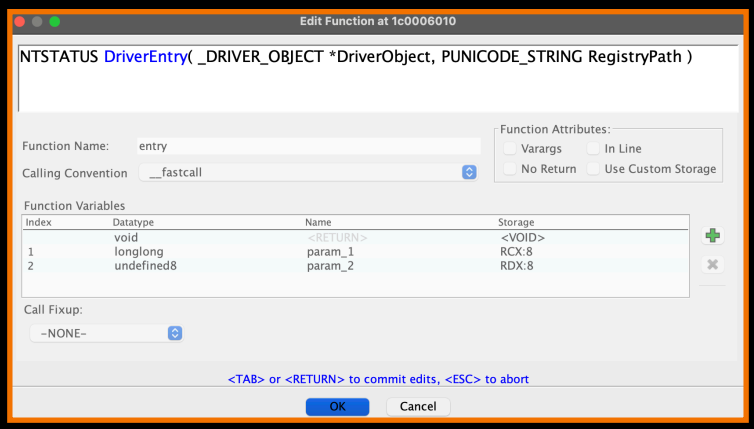
NTSTATUS DriverInitialize(
  [in] _DRIVER_OBJECT *DriverObject,
  [in] PUNICODE_STRING RegistryPath
)
{...}
```

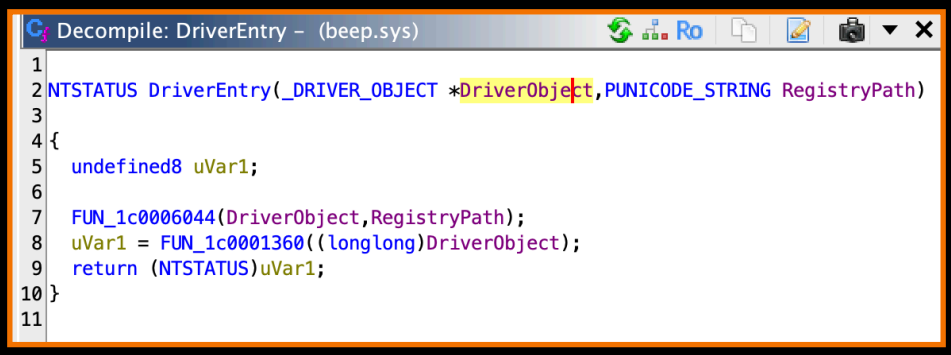
ii. You can right click the function definition line, and select "Edit Function Signature" to change the function's parameters, its name, or return/parameter types.



iii. Open the prompt box and change its contents







iv. After clicking 'OK' and going through some prompts, you should see a change to the decompiled view

v. We have a variable called uVar1 of an unknown type. Let's fix that by first observing that uVar1 is returned on line 9 by DriverEntry which has an NTSTATUS return type (so we know uVar1 is of the type NSTATUS).

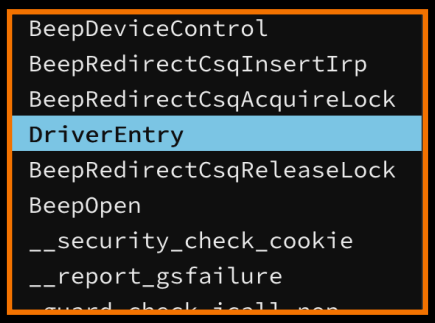
vi. Since this means that FUN_1c0001360 returns an NTSTATUS, edit that function's signature to reflect that fact.

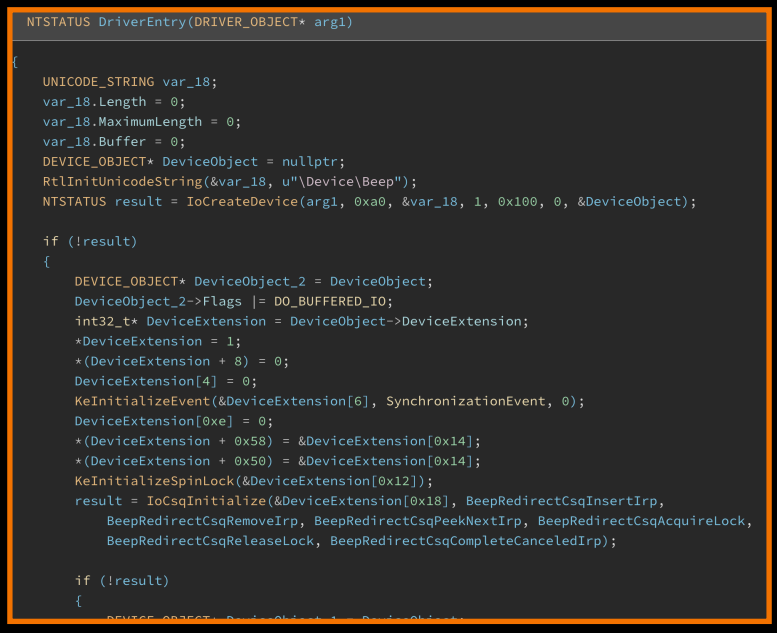
vii. Rename uVar1 to something appropriate

If you have time, take a look into the two functions referenced here within DriverEntry: what do they do, and how do they do it?

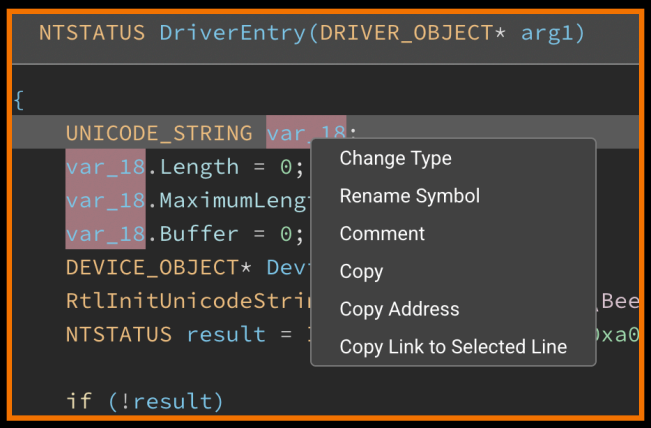
B2: Binary Ninja

1. Find DriverEntry





2. Right click variables and select ‘rename symbol’ or ‘change type’ to adjust the name or type of that variable better match its purpose.



3. Go through the entire function and explore how it works!

IoCreateDevice

This one's simple! Just find the IoCreateDevice call in/near DriverEntry, and list all of the arguments of that specific call and their meanings.
https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-iocreatedevice

G3: Ghidra

No explicit instructions here, *but* if your IoCreateDevice call looks 'funky':

```
18   iVar3 = IoCreateDevice(param_1,0xa0,local_18,1,CONCAT44(uVar4,0x100),
19                          in_stack_fffffffffffffd0 & 0xffffffffffffff00,local_res18);
```

Then use the function signature defined in the above link to edit IoCreateDevice (see G2):

```
14   NVar3 = IoCreateDevice(param_1,0xa0,&local_18,1,0x100,'\0',local_res18);
```

B3: Binary Ninja

See above, except you shouldn't have that weird IoCreateDevice issue in Binary Ninja!

Interrupt Request Packets (IRPs)

Your task is to identify where the IRP handlers are assigned to the device object, and take some time to explore each one of them to better understand how IRPs work.
The most important ones are IRP_MJ_CREATE and IRP_MJ_DEVICE_CONTROL, but check them all if you have time.

https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/irp-major-function-codes

G4: Ghidra

For context, if you are really struggling to find where the IRPs are set, it's in FUN_1c0001360.
The MajorFunction array stores handlers for every possible interrupt request packet type.

```
1c00014d6 0f 1f 44 00 00   NOP          dword ptr [F
1c00014db 33 c0            XOR          EAX,EAX

                       LAB_1c00014dd
1c00014dd 48 8b 5c 24 68   MOV          RBX,qword pt
1c00014e2 48 83 c4 50      ADD          RSP,0x50
1c00014e6 5e               POP          RSI
1c00014e7 c3               RET
1c00014e8 cc               ??           CCh
```

```
29    if (NVar3 == 0) {
30      param_1->DriverStartIo = FUN_1c0001840;
31      param_1->DriverUnload = FUN_1c0001910;
32      param_1->MajorFunction[0] = FUN_1c0001510;
33      param_1->MajorFunction[2] = FUN_1c00011b0;
34      param_1->MajorFunction[0xe] = FUN_1c0001290;
35      param_1->MajorFunction[0x12] = FUN_1c0001070;
36      IoSetStartIoAttributes(local_res18[0],1,0);
37      NVar3 = 0;
38    }
39  }
40  return NVar3;
```

```
DEVICE_OBJECT* DeviceObject_1 = DeviceObject;
arg1->DriverStartIo = BeepStartIo;
arg1->DriverUnload = BeepUnload;
arg1->MajorFunction[0] = BeepOpen;
arg1->MajorFunction[2] = BeepClose;
arg1->MajorFunction[0xe] = BeepDeviceControl;
arg1->MajorFunction[0x12] = BeepCleanup;
IoSetStartIoAttributes(DeviceObject_1, 1, 0);
return STATUS_SUCCESS;
```

B4: Binary Ninja
Binary ninja makes it extremely easy to find where IRP handlers are assigned, but like Ghidra it does not fill in the IRP_MJ_XXX codes so you'll need to do that yourself.

Interrupt Request Packet - Major Function Handlers
All of the IRP handlers follow this structure:
https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nc-wdm-driver_dispatch?source=recommendations

Take your work from G4/B4, and now specifically dig into the handers for IRP_MJ_CREATE and IRP_MJ_DEVICE_CONTROL

G5: Ghidra
While no tool-specific advice is necessary for this, it's worth bearing in mind that you will need to overwrite function definitions/prototypes to match the above structure frequently!

B5: Binary Ninja
See above.

Final Challenge: Triaging Real Vulnerabilities

Your task is to review at least one of the drivers below in comparison to the CVE linked, and determine if they are still vulnerable to the issue presented to NIST!

This is a self-driven challenge intended for completion by individuals at their own pace, but since the drivers in question are either patched or unpatched versions of a once-vulnerable driver, snippets of the reports in question can be found below:

Easy: Beep

Keep digging around the beep.sys driver and see what else you can find! There are no known vulnerabilities in this version.

Medium: MSI Afterburner

https://nvd.nist.gov/vuln/detail/CVE-2024-1460

MSI Afterburner v4.6.5.16370 is vulnerable to a Kernel Memory Leak vulnerability by triggering the 0x80002040 IOCTL code of the RTCore64.sys driver. The handle to the driver can only be obtained from a high integrity process.

Hard: AMD Crash Defender

https://nvd.nist.gov/vuln/detail/CVE-2024-21971

Improper input validation in AMD Crash Defender could allow an attacker to provide the Windows® system process ID to a kernel-mode driver, resulting in an operating system crash, potentially leading to denial of service.

Hard: HTTP Debugger

https://nvd.nist.gov/vuln/detail/CVE-2023-35863

In MADEFORNET HTTP Debugger through 9.12, the Windows service does not set the seclevel registry key before launching the driver. Thus, it is possible for an unprivileged application to obtain a handle to the NetFilterSDK wrapper before the service obtains exclusive access.